

Chapter 33: Graphs in Java

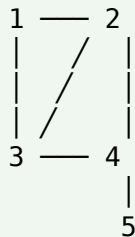
Personal Notes

Trees are strict hierarchies — every node has one parent. But real-world data isn't always that neat. Cities connected by roads. Web pages linked to other pages. Friends on social networks. Course prerequisites. These are all GRAPHS — collections of nodes with connections that can loop, branch, and cross in any pattern.

Graphs are the most versatile data structure in DSA. They're also the last big topic that ties everything together — recursion, queues, stacks, sets, DP — all showing up naturally in graph algorithms.

1. What is a Graph?

A graph is a collection of VERTICES (also called nodes) connected by EDGES. Unlike a tree, edges can go in any direction, form cycles, or leave some vertices completely disconnected.



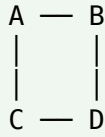
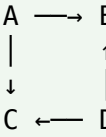
Vertices: {1, 2, 3, 4, 5}
 Edges: {(1,2), (1,3), (2,3), (2,4), (3,4), (4,5)}

Key Terms

Term	Meaning
Vertex (node)	A point in the graph
Edge	A connection between two vertices
Adjacent	Two vertices connected by an edge
Degree	Number of edges touching a vertex
Path	A sequence of vertices connected by edges
Cycle	A path that starts and ends at the same vertex
Connected	There's a path between two vertices
Component	A maximal set of connected vertices

2. Types of Graphs

2.1 Directed vs Undirected

Undirected:	Directed:
	
Edge (A,B) means both A can reach B, and vice versa.	Edge A→B means only from A to B. B might not reach A.

- Undirected — edges have no direction (Facebook friendships)
- Directed — edges have a direction (Twitter follows, course prerequisites)

2.2 Weighted vs Unweighted

- Unweighted — all edges are "equal" (social network friendships)
- Weighted — each edge has a cost/distance/weight (road distances between cities)

2.3 Cyclic vs Acyclic

- Cyclic — contains at least one cycle
- Acyclic — no cycles
- DAG — Directed Acyclic Graph (common for dependencies, scheduling)

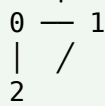
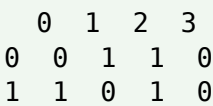
Trees are a special case of graphs: A TREE is a connected, undirected, acyclic graph with N-1 edges for N nodes. Every tree is a graph, but not every graph is a tree. When you learned tree traversals in Ch 30, you were already doing graph algorithms — just on the simplest kind.

3. Representing a Graph in Java

There are two standard ways to store a graph. Each has trade-offs.

3.1 Adjacency Matrix

A 2D array where $\text{matrix}[i][j] = 1$ (or the weight) if there's an edge from i to j, 0 otherwise.

Graph:	Matrix:
	

	2	1	1	0	1
3	3	0	0	1	0

```
int[][] graph = new int[n][n];

// Add an edge (undirected)
graph[u][v] = 1;
graph[v][u] = 1;

// For weighted graphs
graph[u][v] = weight;
```

- Pros: $O(1)$ edge lookup, easy to code
- Cons: $O(n^2)$ space — wasteful for sparse graphs (few edges)
- Use when: small graphs, dense graphs, frequent edge queries

3.2 Adjacency List (Preferred)

An array (or Map) where each vertex has a LIST of its neighbors. Uses only as much space as edges actually exist.

Graph:	Adjacency List:
0 — 1	0 → [1, 2]
/	1 → [0, 2]
2	2 → [0, 1, 3]
	3 → [2]
3	

```
import java.util.*;

List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

// Add edge (undirected)
graph.get(u).add(v);
graph.get(v).add(u);

// For weighted graphs – use Pair or int[]
// graph.get(u).add(new int[]{v, weight});
```

- Pros: $O(V + E)$ space, efficient iteration over neighbors
- Cons: $O(\text{degree})$ to check if a specific edge exists
- Use when: most real-world graphs (which are usually sparse)

The default choice: Adjacency list is preferred for 90% of interview problems. It's cheaper for typical graphs (many nodes, few edges) and matches the way you'll iterate through neighbors during DFS/BFS. Only reach for a matrix if edge lookups dominate or the graph is truly dense.

4. DFS — Depth-First Search

DFS explores as far as possible along each branch before backtracking. It's the same idea as tree DFS from Ch 30 — but graphs can have CYCLES, so you must track visited vertices.

Recursive DFS

```
public static void dfs(int node, List<List<Integer>> graph, boolean[] visited) {
    visited[node] = true;
    System.out.print(node + " ");

    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            dfs(neighbor, graph, visited);
        }
    }
}

// Usage:
boolean[] visited = new boolean[n];
dfs(0, graph, visited);
```

Iterative DFS (Using a Stack)

```
public static void dfsIterative(int start, List<List<Integer>> graph) {
    boolean[] visited = new boolean[graph.size()];
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(start);

    while (!stack.isEmpty()) {
        int node = stack.pop();
        if (visited[node]) continue;
        visited[node] = true;
        System.out.print(node + " ");

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) stack.push(neighbor);
        }
    }
}
```

The visited array is critical: Without it, graph DFS runs FOREVER on any graph with a cycle. Tree DFS didn't need it because trees have no cycles. This is the single biggest difference between tree DFS and graph DFS — always track visited nodes.

5. BFS — Breadth-First Search

BFS explores the graph LEVEL BY LEVEL, visiting all vertices at distance 1 before those at distance 2, and so on. This makes BFS the natural choice for SHORTEST PATH in unweighted graphs.

Code — Uses a Queue

```
public static void bfs(int start, List<List<Integer>> graph) {
    boolean[] visited = new boolean[graph.size()];
    Queue<Integer> queue = new LinkedList<>();

    queue.offer(start);
    visited[start] = true;

    while (!queue.isEmpty()) {
        int node = queue.poll();
        System.out.print(node + " ");

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
}
```

DFS vs BFS at a Glance

Aspect	DFS	BFS
Data structure	Stack (or recursion)	Queue
Order visited	Depth-first — goes deep first	Level by level
Shortest path?	No (unweighted graphs)	Yes (unweighted graphs)
Space	$O(H)$ — deepest path	$O(W)$ — widest level
Best for	Cycle detection, paths, backtrack	Shortest distance, level info

6. Common Graph Problems

6.1 Count Connected Components

In an undirected graph, how many separate "islands" are there? Run DFS from each unvisited node — every time you start a fresh DFS, you've found a new component.

```

public static int countComponents(int n, List<List<Integer>> graph) {
    boolean[] visited = new boolean[n];
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i, graph, visited);
            count++;
        }
    }
    return count;
}

```

6.2 Shortest Path (Unweighted)

BFS naturally finds the shortest path in an unweighted graph — the first time you reach a node is via the shortest path.

```

public static int shortestPath(int start, int end, List<List<Integer>>
graph) {
    if (start == end) return 0;
    boolean[] visited = new boolean[graph.size()];
    Queue<int[]> queue = new LinkedList<>();

    queue.offer(new int[]{start, 0});
    visited[start] = true;

    while (!queue.isEmpty()) {
        int[] curr = queue.poll();
        int node = curr[0], dist = curr[1];

        for (int neighbor : graph.get(node)) {
            if (neighbor == end) return dist + 1;
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(new int[]{neighbor, dist + 1});
            }
        }
    }
    return -1;    // no path
}

```

6.3 Cycle Detection (Undirected Graph)

During DFS, if you encounter a visited neighbor that isn't your immediate parent, there's a cycle.

```

public static boolean hasCycle(List<List<Integer>> graph) {
    int n = graph.size();
    boolean[] visited = new boolean[n];

```

```

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                if (dfsCycle(i, -1, graph, visited)) return true;
            }
        }
        return false;
    }

    static boolean dfsCycle(int node, int parent, List<List<Integer>> graph,
        boolean[] visited) {
        visited[node] = true;
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                if (dfsCycle(neighbor, node, graph, visited)) return true;
            } else if (neighbor != parent) {
                return true;    // visited AND not parent → cycle
            }
        }
        return false;
    }
}

```

6.4 Number of Islands (Grid DFS)

Given a 2D grid of '1' (land) and '0' (water), count the number of islands. Every unvisited '1' starts a new island — flood-fill via DFS.

```

public static int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') {
                dfsIsland(grid, r, c);
                count++;
            }
        }
    }
    return count;
}

static void dfsIsland(char[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (grid[r][c] != '1') return;

    grid[r][c] = '0';    // mark visited
    dfsIsland(grid, r + 1, c);
    dfsIsland(grid, r - 1, c);
    dfsIsland(grid, r, c + 1);
    dfsIsland(grid, r, c - 1);
}

```

```
}
```

Grids ARE graphs: A 2D grid is just a graph where each cell is a vertex and its neighbors are the 4 (or 8) adjacent cells. Many "grid" problems in interviews are graph problems in disguise — flood fill, shortest path in a maze, surrounded regions. Same DFS/BFS templates work directly.

7. Topological Sort (DAG only)

For a Directed Acyclic Graph (DAG), a topological sort is a linear ordering of vertices such that every edge ($u \rightarrow v$) has u appearing BEFORE v . Useful for task scheduling, course prerequisites, build systems.

Example — Course Schedule

Prerequisites:	Valid topological orders:
Math $\rightarrow$ Physics	Math, Physics, Chemistry, Biology
Math $\rightarrow$ Chemistry	Math, Chemistry, Physics, Biology
Physics $\rightarrow$ Biology	
Chemistry $\rightarrow$ Biology	(both are valid – multiple orderings exist)

Kahn's Algorithm (BFS-based)

```
public static List<Integer> topologicalSort(int n, List<List<Integer>>
graph) {
    int[] inDegree = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : graph.get(u))
            inDegree[v]++;

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++)
        if (inDegree[i] == 0) queue.offer(i);

    List<Integer> result = new ArrayList<>();
    while (!queue.isEmpty()) {
        int node = queue.poll();
        result.add(node);
        for (int neighbor : graph.get(node)) {
            if (--inDegree[neighbor] == 0) queue.offer(neighbor);
        }
    }

    return result.size() == n ? result : new ArrayList<>(); // cycle →
empty
}
```

Kahn's algorithm ALSO detects cycles: if the result doesn't include all nodes, there's a cycle (nodes stuck with non-zero in-degree).

8. Dijkstra's Algorithm (Shortest Path in Weighted Graph)

BFS finds the shortest path when all edges are equal. But when edges have DIFFERENT weights (e.g., road distances), we need Dijkstra's algorithm. It uses a priority queue to always process the closest unvisited node.

```
public static int[] dijkstra(int start, List<List<int[]>> graph, int n) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[start] = 0;

    // PriorityQueue sorted by distance
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{start, 0});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int node = curr[0], d = curr[1];

        if (d > dist[node]) continue;    // outdated entry

        for (int[] edge : graph.get(node)) {
            int neighbor = edge[0], weight = edge[1];
            int newDist = d + weight;

            if (newDist < dist[neighbor]) {
                dist[neighbor] = newDist;
                pq.offer(new int[]{neighbor, newDist});
            }
        }
    }
    return dist;
}
```

Dijkstra limitation: Dijkstra ONLY works with NON-NEGATIVE edge weights. If your graph has negative weights, use the Bellman-Ford algorithm instead. And Dijkstra doesn't handle negative CYCLES at all — the whole idea of "shortest path" breaks.

9. Graph Traversal Complexity

Operation	Time	Space
DFS	$O(V + E)$	$O(V)$ for visited + stack
BFS	$O(V + E)$	$O(V)$ for visited + queue
Topological Sort	$O(V + E)$	$O(V)$

Operation	Time	Space
Dijkstra (with heap)	$O((V+E) \log V)$	$O(V)$
Adjacency Matrix ops	$O(V^2)$ traversal	$O(V^2)$

V = number of vertices, E = number of edges. $O(V + E)$ traversals are optimal — every vertex and edge is touched at most a constant number of times.

10. When to Reach for Which Algorithm

If you need to...	Use
Visit every node / explore all paths	DFS or BFS
Shortest path in unweighted graph	BFS
Shortest path in weighted graph (non-negative)	Dijkstra
Detect cycle in undirected graph	DFS with parent tracking
Detect cycle in directed graph	DFS with 3-color states, or Kahn's
Count connected components	DFS or BFS, count restarts
Ordering with dependencies	Topological sort (Kahn's or DFS)
Flood fill / count islands / regions	DFS or BFS on grid
Bipartite check (2-color problem)	BFS with alternating colors

11. Common Mistakes

Mistake 1: Forgetting the visited array

```
// Infinite loop on any graph with a cycle:
void dfs(int node, List<List<Integer>> graph) {
    for (int n : graph.get(node)) dfs(n, graph);    // x
}

// Always track visited:
void dfs(int node, boolean[] visited, ...) {
    if (visited[node]) return;
    visited[node] = true;
    ...
}
```

Mistake 2: Adding edge only in one direction (undirected graph)

```
// For an UNDIRECTED graph, always add both directions:
graph.get(u).add(v);
graph.get(v).add(u);      // don't forget this!
```

Mistake 3: Using DFS for unweighted shortest path

DFS goes deep first — the first time it reaches a target is NOT necessarily the shortest. Use BFS instead for shortest path in unweighted graphs.

Mistake 4: Using Dijkstra with negative weights

Dijkstra assumes visiting a node with the smallest known distance means we've found the shortest path to it. With negative weights, a later path might be shorter — so Dijkstra produces wrong answers. Use Bellman-Ford instead.

Mistake 5: Marking visited at wrong time in BFS

```
// WRONG — mark when polling. Same node gets added multiple times.
while (!queue.isEmpty()) {
    int node = queue.poll();
    visited[node] = true;                // x too late
    for (int n : graph.get(node)) queue.offer(n);
}

// RIGHT — mark when adding to queue
if (!visited[neighbor]) {
    visited[neighbor] = true;           // ✓ mark on enqueue
    queue.offer(neighbor);
}
```

Mistake 6: Confusing directed and undirected

For directed graphs, edge (u, v) does NOT imply (v, u). Read the problem carefully — is the relationship symmetric or asymmetric? Friends vs Twitter follows is the classic distinction.

Mistake 7: StackOverflowError on huge graphs

Recursive DFS goes stack-deep. For very large graphs (millions of nodes) or highly-connected ones, convert to iterative DFS with an explicit stack.

12. Best Practices

- Default to adjacency list — it's cheaper and matches DFS/BFS naturally
- Always maintain a visited[] array (or Set) — graphs have cycles, trees don't
- For undirected graphs, add each edge from BOTH directions when building
- Use BFS for shortest path in unweighted graphs, Dijkstra for weighted
- Mark visited when ENQUEUEING in BFS (not when polling) to avoid duplicates

- For grids, use direction arrays: `int[] dr = {-1, 1, 0, 0}; int[] dc = {0, 0, -1, 1};`
- For topological sort, prefer Kahn's algorithm — it also detects cycles
- For very large graphs, convert recursive DFS to iterative to avoid stack overflow

13. Quick Reference

Concept	Meaning
Vertex / Node	A point in the graph
Edge	Connection between two vertices
Adjacency Matrix	2D array — <code>matrix[i][j]</code> = edge between i and j
Adjacency List	Array of lists — <code>list[i]</code> = neighbors of i
DFS	Depth-first, uses stack/recursion
BFS	Breadth-first, uses queue
Visited	Track visited nodes to avoid infinite loops
Connected Component	Group of vertices all reachable from each other
Topological Sort	Linear ordering of DAG respecting all edges
Kahn's Algorithm	BFS-based topological sort (also detects cycles)
Dijkstra	Shortest path with non-negative weights
DAG	Directed Acyclic Graph
Bipartite	Graph 2-colorable so no edge connects same colors

14. Practice Problems

Try in order — each builds on techniques from the previous.

Warm-ups

1. Build an adjacency list from an edge list `[(0,1), (0,2), (1,2), (2,3)]`. Print it.
2. Print all nodes reachable from node 0 using DFS.
3. Print all nodes reachable from node 0 using BFS.
4. Count the number of connected components in an undirected graph.

Path Problems

5. Find the shortest path from A to B in an unweighted graph (BFS).
6. Check if there's a path from A to B (DFS or BFS).
7. Find all paths from A to B in a DAG.
8. Detect a cycle in an undirected graph.
9. Detect a cycle in a directed graph.

Grid Problems

10. Number of Islands — count connected regions of '1' in a 2D grid.
11. Max Area of Island — find the largest island by area.
12. Rotting Oranges — BFS from multiple sources; time until all oranges rot.
13. Flood Fill — recolor connected pixels of the same color.
14. Surrounded Regions — flip all 'O's not connected to the boundary.

Ordering & Advanced

15. Course Schedule — can you finish all courses given prerequisites? (Cycle detection)
16. Course Schedule II — return one valid course order (topological sort).
17. Is Graph Bipartite? — BFS with 2-coloring.
18. Word Ladder — shortest transformation from beginWord to endWord (BFS).
19. Clone Graph — deep copy of a graph using BFS or DFS.
20. Network Delay Time — Dijkstra's algorithm application.
21. Cheapest Flights Within K Stops — modified Dijkstra / Bellman-Ford.

15. Final Takeaway

Graphs unify almost everything you've learned. DFS is recursion. BFS uses queues. Topological sort uses in-degree counting. Dijkstra uses a priority queue (min-heap). Grid problems are graphs in disguise. Once you see the general pattern — vertices + edges + traversal — the specific problem starts to look familiar.

For interviews, master the two traversals (DFS and BFS) and understand when each applies. Then layer on cycle detection, topological sort, and Dijkstra. That toolkit handles 90% of graph interview questions — from social networks to grid problems to build systems.

Key Takeaway: Graphs = vertices + edges. Use adjacency list. Always track visited. DFS for exploration and cycle detection; BFS for shortest paths in unweighted graphs; Dijkstra for weighted; Kahn's for topological order. Master these five algorithms and the vast majority of graph problems become approachable. Recognize that grids and dependencies are graphs in disguise.

16. What's Next?

With graphs done, the DSA toolkit is nearly complete. What's left:

- Heaps & Priority Queues in depth — top-K problems, Dijkstra's engine
- Tries — prefix trees for word search and autocomplete
- Union-Find (Disjoint Set Union) — fast connectivity queries
- Advanced Graphs — Minimum Spanning Tree (Kruskal, Prim), Bellman-Ford
- Segment Tree / Fenwick Tree — range queries and updates
- Bit Manipulation — fast tricks using binary representation
- Sliding Window — efficient subarray/substring patterns
- System Design — for later interview rounds